BSc (Hons) in **Information Technology Specialized in AI**

Faculty of Computing

IT3012 : Intelligent Agents

Year 3 · Semester 1 · 2026 Semester 2

Practical 01

> **Objective & Lecture Mapping:** Recall that an intelligent agent acts within a continuous loop: Sensors generate percept sequences, and Actuators execute actions to maximize a Performance measure. Use this sheet to execute practical modifications and incrementally evaluate your design against Lecture 01 and 02 theory (from basic recall to analytical classification).

Part 1: Code Analysis & PEAS Evaluation

Task 0 : Setting up and getting the starter Code

1. Go to the Git Hub Repo. Create a Fork of this Git Repo to your Personal Github Profile. Open it using your favourite IDE and follow the below instruction to complete the practical. The base code is in the "master" brach of the repo.

Task 1.1: The Environment State (__init__)

1. (Remember) List the four components of the PEAS framework discussed in Lecture 01.

P - Performance Measure: The goal or metric used to evaluate how successfully the agent performs (e.g., total game score).

E - Environment: The external world or setting in which the agent operates (e.g., grid size, walls, food locations).
A - Actuators: The mechanisms or tools the agent uses to perform actions on the environment (moving Up, Down, Left, or Right).

S - Sensors: The devices or functions used by the agent to observe/perceive the environment.

2. (Understand) Look at the `VisualGridHuntGame.__init__` method. Which specific variables in this code represent the physical "Environment (E)" state?

self.width and self.height: Define the grid dimensions/boundaries.
self.walls: Holds coordinates of static wall obstacles.
self.food_positions: Holds coordinates where food pellets exist.
self.opponents: Holds positions of adversarial moving entities.
self.agent_pos: Represents the physical coordinate position of the agent in the grid world.

3. (Analyze) Based on the variables initialized (specifically `self.opponents`), classify this baseline environment as "Single-Agent" or "Multi-Agent". Briefly justify your choice.

Classification: Multi-Agent environment.

Justification: The environment contains self.opponents that move independently and interact with the main agent (causing score penalties on collision). Because the state and outcome depend on the actions of multiple entities, it is classified as multi-agent.

Task 1.2: The Perception Subsystem (get_percept)

4. (Remember) Define what a "percept sequence" is according to Lecture 01.

A percept sequence is the complete historical sequence of all sensory inputs the agent has observed from the start of its operation up to the current moment.

5. (Understand) Which component of the PEAS framework does the `get_percept()` method represent in our code?

It represents the Sensors (S) component. It acts as the agent's "eyes and ears" by retrieving and structuring data from the environment into a dictionary.

6. (Evaluate) Based on the exact dictionary returned by `get_percept()`, is this environment "Fully Observable" or "Partially Observable"? Explain why based on what the agent can and cannot see.

Classification: Fully Observable.
Explanation: The dictionary returned by get_percept() exposes the exact global state—including the agent's exact location, all opponent locations, food presence, wall hits, score, and remaining food count. There are no unobserved or hidden variables in the environment.

Task 1.3: Action Execution (execute_action)

7. (Understand) When `execute_action()` deducts points for hitting a wall, which PEAS component is being actively updated?

It updates the Performance Measure (P) component by subtracting 5 points from self.score

8. (Evaluate) Why is it structurally important that this metric evaluates changes in the external environment state (hitting a wall) rather than the agent's internal processing effort?

Performance metrics must measure actual real-world outcomes, not the agent's internal computing effort. If internal effort were rewarded, an agent could achieve a high score simply by looping endlessly in thought without taking meaningful actions in its environment.

Part 2: Guided Modification & Deep Theory Mapping

> Follow the implementation instructions to inject a new hazard, then answer the questions to map your code changes back to the theoretical nature of the environment.

Step 2.1: Extending the Environment Initialization (__init__)

1. **Implementation Instructions:** Declare a new trap collection attribute (e.g., `self.toxic_traps = set()`) inside `__init__`. Populate it with coordinate tuples safely avoiding position `(0, 0)`, walls, and food.

9. (Understand) If you successfully add `self.toxic_traps` to the environment but intentionally hide this data from the agent's sensors, how does this specifically alter the environment's "Observability" classification?

The environment shifts from Fully Observable to Partially Observable. Because toxic traps exist in the physical environment state but are intentionally hidden from get_percept(), the agent can no longer see the complete state of the world.

Step 2.2: Updating the Perception Subsystem (get_percept)

1. **Implementation Instructions:** Locate `get_percept(self)` and add a new boolean sensor key (e.g., `'smells_toxin': tuple(self.agent_pos) in self.toxic_traps`) to the dictionary returned to the agent loop.

10. (Analyze) By adding `'smells_toxin'`, you expand the agent's percept sequence. Explain how this specific sensor helps the agent act with "Rationality" (maximizing expected utility).

Rationality means choosing actions that maximize the expected performance measure based on available sensory information. By adding 'smells_toxin', the agent senses when it is on a dangerous tile. This extra information allows the agent to make rational decisions (such as moving away immediately) to prevent severe score penalties (-15 points)

Step 2.3: Modifying Action Execution & Visual Rendering (execute_action & draw_grid)

1. **Implementation Instructions:** In `execute_action`, check if the updated `self.agent_pos` intersects with `self.toxic_traps` to decrement `self.score` by 15 points. In `draw_grid`, iterate through traps to render custom purple shapes on the canvas.

11. (Remember) You programmed a severe score penalty for stepping on a trap to avoid metric exploitation. What was the classic "Vacuum World Exploit" example discussed in Lecture 01 that illustrated the danger of faulty metrics?

The Exploit: In the classic Vacuum World example, if the performance metric awards 1 point every time the vacuum cleans dirt, a naive agent can dump dirt back onto a clean floor and clean it again repeatedly to earn infinite points.

Key Takeaway: Performance metrics must be designed carefully to reward true goal completion rather than rewarding intermediate actions that can be exploited in endless loops.

Finalize and Lock Lab 01 Analytical Evaluation

Lab 01 code analysis and theoretical evaluation complete and verified!